Implementation Security

ทฤษฎีคณิตศาสตร์พื้นฐานของ ECC มีระบบระรักษาความปลอดภัยที่แข็งแรงมาตลอด การที่ระบบ
คริปโตถูกแฮก แทบจะเกิดจากความผิดพลาดในขั้นตอน “การนำไปเขียนโค้ดหรือสร้างระบบจริง” ทั้งสิ้น โดยข้อผิดพลาดที่เจอบ่อยมีดังนี้ :

- โดนจับผิดพฤติกรรมการทำงาน (Timing และ Side channels)

หากโค้ดที่เราเขียนมามีการใช้คำสั่ง หรือใช้เวลาคำนวณช้า-เร็วไม่เท่ากัน (Variable-time) แฮกเกอร์สามารถเดา “Private key” ได้จากการจับเวลาที่คอมพิวเตอร์ใช้ประมวลผล (Timing) แอบวัดอัตราการกินไฟ หรือดูดคลื่นแม่เหล็กไฟฟ้าที่แผ่ออกมา

วิธีแก้: ต้องเขียนโค้ดแบบใช้เวลาทำงานเท่ากันเสมอ รวมถึงใช้เทคนิคทางคณิตศาสตร์ เช่น Montgomery laadder และการสุ่มเพื่อพรางข้อมูลไม่ให้เดาได้

- โดนหลอกด้วยข้อมูลนอกเส้นโค้ง (Invalid-curve)

ถ้าเราเขียนโปรแกรมรับข้อมูลเข้ามา โดยไม่ตรวจเช็กให้ดีว่า “จุด” ที่รับเข้ามาอยู่บนเส้นโค้งจริงๆ หรือไม่ แฮกเกอร์อาจจะจงใจส่งข้อมูลของเส้นโค้งอื่นที่อ่อนแอกว่ามาให้ให้ระบบคำนวณ ซึ่งทำให้แฮกเกอร์แงะ Privaate key ได้ทีละส่วน (ระบบ TLS จริงๆ เคยโดนเจาะด้วยวิธีนี้มาแล้ว)

วิธีแก้: ต้องตรวจสอบ ทุกข้อมูลที่รับเข้ามาเสมอ หรือเปลี่ยนไปใช้เส้นโค้งที่มีการออกแบบมา
เพื่อป้องกันเรื่องนี้แต่แรก อย่างเช่น Curve25519

- **ระบบสุ่มเลขมีปัญหาตอนเซ็นชื่อ (Nonce failures)**

ในการสร้างลายเซ็นดิจิทัล (ECDSA) ระบบจะต้องใช้ตัวเลขสุ่มที่เรียกว่า Nonce หากระบบสุ่มเลขออกมาซ้ำกันเดาทางได้ หรือสุ่มไม่ดี Privaate key ของเราจะถูกเปิดเผยให้แฮกเกอร์รู้

วิธีแก้: เลิกพึ่งพาระบบสุ่มเลขในจังหวะที่ทำการเซ็น แต่เปลี่ยนไปใช้วิธีสร้าง Nonce แบบคำนวณตายตัว (Deterministic nonces) ตามมาตรฐาน เช่น RFC 6979 หรือ EdDSA แทน

- ความสับสนเรื่องรูปแบบข้อมูล (Cofator confusion)

เส้นโค้งบางแบบยอมให้ข้อมูลลายเซ็นหรือจุด “ตัวเดียวกัน” สามารถเขียนออกมาในรูปแบบ (Encodings) ที่หน้าตาต่างกันได้ เช่น คำว่า “รถยนต์” กับ “รถ” ที่ความหมายเดียวกันแต่เขียนต่างกัน สิ่งนี้ทำให้ระบบที่คิดเอาเองว่ารูปแบบข้อมูลต้องมีแค่อย่างเดียว ทำงานรวนหรือเกิดบั๊กได้

วิธีแก้: ใช้เครื่องมือช่วย เพื่อบังคับให้ข้อมูลอ่านได้รูปแบบเดียวอย่างชัดเจน เช่น Ristretto255

- แกล้งทำให้ฮาร์ดแวร์ทำงานรวน (Fault attacks)

แฮกเกอร์อาจโจมตีทางฮาร์แวร์เช่นแกล้งกระตุกกระแสไฟ (Voltageglitching) ขณะที่อุปกรณ์กำลังคำนวณรหัสผ่านเพื่อทำให้ฮาร์ดแวร์ทำงานเพี้ยนจนรหัสหลุดออก

วิธีแก้: อุปกรณ์อย่างสมาร์ตการ์จะต้องออกแบบให้ตรวจสอบผลลัพธ์การนำควณของตัวเอง
ก่อนส่งข้อมูลออกไป

การเขียนโค้ดระบบ ECC ให้ปลอดภัยนั้นมีรายละเอียดจุกจิกที่ต้องระวังเยอะมาก ข้อแนะนำคือ “อย่าเขียนโค้ดเข้ารหัสขึ้นมาใช้งานเอง” (Custom code) แต่ควรไปหยิบเอาไลบรารีที่ผู้เชี่ยวชาญสร้าง
และตรวจสอบมาอย่างดีมาใช้งานแทนจะปลอดภัยกว่ามาก